ECA5801-IOE AND AI DRIVEN SYSTEM

SIMULATION

EXPERIMENT 12

K.POOJA SREE

192512245

Simulation and Performance Evaluation of Cross-Platform Edge AI Model Deployment using ONNX and PyTorch Mobile

```
# ============================================================
# SIMULATION EXPERIMENT 10
# Cross-Platform Edge AI Deployment using ONNX and PyTorch Mobile
# Google Colab | Complete single-cell experiment
# ============================================================

import os
import sys
import time
import subprocess
import urllib.request
import tarfile
import gc
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")

# -------------------- 1. INSTALL DEPENDENCIES --------------------

print("Installing ONNX dependencies...")
subprocess.check_call([
    sys.executable, "-m", "pip", "install", "-q",
```

```
    "onnx", "onnxruntime", "onnxscript"
])

import torch
import torch.nn as nn
import torchvision
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
from torchvision import transforms
from PIL import Image
import onnx
import onnxruntime as ort

try:
    from onnxruntime.quantization import quantize_dynamic, QuantType
    ONNX_QUANT_AVAILABLE = True
except Exception:
    ONNX_QUANT_AVAILABLE = False

try:
    from torch.utils.mobile_optimizer import optimize_for_mobile
    MOBILE_OPT_AVAILABLE = True
except Exception:
    MOBILE_OPT_AVAILABLE = False

# ---------- 2. CONFIGURATION ----------

SEED = 42
NUM_IMAGES = 100
WARMUP_RUNS = 10
TIMED_RUNS = 100
THREAD_COUNT = 2
IMAGE_SIZE = 224
```

```
OPSET = 13

OUT_DIR = "/content/edge_ai_experiment"
DATA_DIR = "/content/imagenette2-160"
ARCHIVE = "/content/imagenette2-160.tgz"

os.makedirs(OUT_DIR, exist_ok=True)
torch.manual_seed(SEED)
np.random.seed(SEED)

# FIX: Inter-op threads can only be set before parallel work starts.
# This try/except prevents Colab's RuntimeError on repeated runs.
try:
    torch.set_num_interop_threads(1)
except RuntimeError:
    print("Inter-op threads already initialized; keeping current setting.")

torch.set_num_threads(THREAD_COUNT)
device = torch.device("cpu")

print("\nPyTorch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("ONNX:", onnx.__version__)
print("ONNX Runtime:", ort.__version__)
print("Execution device:", device)
print("Intra-op thread count:", torch.get_num_threads())

# ---------- 3. DOWNLOAD TEST DATA ----------

IMAGENETTE_URL = (
    "https://s3.amazonaws.com/fast-ai-imageclas/imagenette2-160.tgz"
)
```

```
if not os.path.isdir(os.path.join(DATA_DIR, "val")):
    print("\nDownloading Imagenette dataset...")
    urllib.request.urlretrieve(IMAGENETTE_URL, ARCHIVE)

    print("Extracting dataset...")
    with tarfile.open(ARCHIVE, "r:gz") as tar:
        # Dataset archive from the official fast.ai public URL.
        tar.extractall("/content")

    if os.path.exists(ARCHIVE):
        os.remove(ARCHIVE)

VAL_DIR = os.path.join(DATA_DIR, "val")

# Imagenette class folders and their corresponding ImageNet labels
SYNSET_TO_IMAGENET = {
    "n01440764": 0,
    "n02102040": 217,
    "n02979186": 482,
    "n03000684": 491,
    "n03028079": 497,
    "n03394916": 566,
    "n03417042": 569,
    "n03425413": 571,
    "n03445777": 574,
    "n03888257": 701
}

preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(IMAGE_SIZE),
```

```
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

image_paths, labels = [], []

for folder, class_index in SYNSET_TO_IMAGENET.items():
    folder_path = os.path.join(VAL_DIR, folder)

    if not os.path.isdir(folder_path):
        continue

    files_in_folder = sorted([
        os.path.join(folder_path, filename)
        for filename in os.listdir(folder_path)
        if filename.lower().endswith((".jpg", ".jpeg", ".png"))
    ])

    for path in files_in_folder:
        image_paths.append(path)
        labels.append(class_index)

if not image_paths:
    raise RuntimeError(
        "Validation images not found. Check dataset download/extraction."
    )

# Deterministically choose up to NUM_IMAGES, balanced across classes
rng = np.random.default_rng(SEED)
```

```
selected_indices = []

for class_index in SYNSET_TO_IMAGENET.values():
    class_indices = [
        i for i, label in enumerate(labels) if label == class_index
    ]
    selected_indices.extend(class_indices[:NUM_IMAGES // 10])

selected_indices = selected_indices[:NUM_IMAGES]

if len(selected_indices) < min(NUM_IMAGES, len(image_paths)):
    selected_set = set(selected_indices)
    for i in range(len(image_paths)):
        if i not in selected_set:
            selected_indices.append(i)
        if len(selected_indices) >= NUM_IMAGES:
            break

image_paths = [image_paths[i] for i in selected_indices]
labels = np.array([labels[i] for i in selected_indices], dtype=np.int64)

images = []
for path in image_paths:
    with Image.open(path) as image:
        images.append(preprocess(image.convert("RGB")))

x_test = torch.stack(images).contiguous()
y_test = torch.tensor(labels, dtype=torch.long)

print("\nTest images:", len(x_test))
print("Input shape:", tuple(x_test.shape))
```

```
# ---------- 4. LOAD BASELINE MODEL ----------

print("\nLoading pretrained MobileNetV2...")
weights = MobileNet_V2_Weights.DEFAULT
model = mobilenet_v2(weights=weights).eval().to(device)
dummy_input = torch.randn(1, 3, IMAGE_SIZE, IMAGE_SIZE)

with torch.inference_mode():
    baseline_logits = model(x_test)

baseline_predictions = baseline_logits.argmax(dim=1)
baseline_accuracy = (
    (baseline_predictions == y_test).float().mean().item() * 100
)

print(f"PyTorch baseline accuracy: {baseline_accuracy:.2f}%")

# ---------- 5. BENCHMARK FUNCTIONS ----------

def benchmark_torch(call, inputs):
    for _ in range(WARMUP_RUNS):
        with torch.inference_mode():
            call(inputs[:1])

    times = []
    with torch.inference_mode():
        for _ in range(TIMED_RUNS):
            start = time.perf_counter()
            call(inputs[:1])
            times.append((time.perf_counter() - start) * 1000)

    return {
```

```
        "mean": float(np.mean(times)),
        "std": float(np.std(times)),
        "p95": float(np.percentile(times, 95)),
        "fps": float(1000 / np.mean(times))
    }


def benchmark_onnx(session, sample):
    input_name = session.get_inputs()[0].name

    def run(arr):
        return session.run(None, {input_name: arr})

    for _ in range(WARMUP_RUNS):
        run(sample)

    times = []
    for _ in range(TIMED_RUNS):
        start = time.perf_counter()
        run(sample)
        times.append((time.perf_counter() - start) * 1000)

    return {
        "mean": float(np.mean(times)),
        "std": float(np.std(times)),
        "p95": float(np.percentile(times, 95)),
        "fps": float(1000 / np.mean(times))
    }


def evaluate_torch(call, inputs, targets):
    outputs = []
```

```
    with torch.inference_mode():
        for i in range(len(inputs)):
            outputs.append(call(inputs[i:i+1]).detach().cpu())

    logits = torch.cat(outputs, dim=0)
    accuracy = (
        (logits.argmax(dim=1) == targets).float().mean().item() * 100
    )

    return logits, float(accuracy)


def evaluate_onnx(session, inputs, targets):
    input_name = session.get_inputs()[0].name
    outputs = []

    for i in range(len(inputs)):
        array = inputs[i:i+1].numpy().astype(np.float32)
        outputs.append(session.run(None, {input_name: array})[0])

    logits = np.concatenate(outputs, axis=0)
    accuracy = (
        (logits.argmax(axis=1) == targets.numpy()).mean() * 100
    )

    return logits, float(accuracy)


def size_mb(path):
    return os.path.getsize(path) / (1024 * 1024)
```

```
# -------------------- 6. PYTORCH BASELINE ------------------------

baseline_path = os.path.join(OUT_DIR, "mobilenet_v2_fp32.pth")
torch.save(model.state_dict(), baseline_path)

baseline_timing = benchmark_torch(model, x_test)
baseline_size = size_mb(baseline_path)

print("\nBaseline latency:", round(baseline_timing["mean"], 3), "ms")
print("Baseline model size:", round(baseline_size, 3), "MB")

# -------------------- 7. EXPORT ONNX FP32 ------------------------

onnx_fp32_path = os.path.join(OUT_DIR, "mobilenet_v2_fp32.onnx")

print("\nExporting ONNX FP32 using the legacy exporter...")

# dynamo=False avoids the new exporter path that caused onnxscript errors
torch.onnx.export(
    model,
    dummy_input,
    onnx_fp32_path,
    export_params=True,
    opset_version=OPSET,
    do_constant_folding=True,
    input_names=["input"],
    output_names=["output"],
    dynamic_axes={
        "input": {0: "batch_size"},
        "output": {0: "batch_size"}
    },
```

```
    dynamo=False
)

onnx.checker.check_model(onnx.load(onnx_fp32_path))
print("ONNX checker: PASSED")

session_options = ort.SessionOptions()
session_options.intra_op_num_threads = THREAD_COUNT
session_options.inter_op_num_threads = 1

onnx_session = ort.InferenceSession(
    onnx_fp32_path,
    sess_options=session_options,
    providers=["CPUExecutionProvider"]
)

onnx_logits, onnx_accuracy = evaluate_onnx(
    onnx_session, x_test, y_test
)

onnx_diff = float(np.max(np.abs(
    baseline_logits.cpu().numpy() - onnx_logits
)))

onnx_timing = benchmark_onnx(
    onnx_session, x_test[:1].numpy().astype(np.float32)
)

print("ONNX output difference:", f"{onnx_diff:.8g}")
print("ONNX accuracy:", round(onnx_accuracy, 2), "%")

# ------------------- 8. ONNX INT8 ------------------------------
```

```
onnx_int8_path = os.path.join(OUT_DIR, "mobilenet_v2_int8.onnx")
onnx_int8_session = None
onnx_int8_accuracy = None
onnx_int8_timing = None
onnx_int8_diff = None
onnx_int8_status = "Not run"

if ONNX_QUANT_AVAILABLE:
    try:
        print("\nQuantizing ONNX model to dynamic INT8...")

        quantize_dynamic(
            model_input=onnx_fp32_path,
            model_output=onnx_int8_path,
            weight_type=QuantType.QInt8
        )

        onnx_int8_session = ort.InferenceSession(
            onnx_int8_path,
            sess_options=session_options,
            providers=["CPUExecutionProvider"]
        )

        q_logits, onnx_int8_accuracy = evaluate_onnx(
            onnx_int8_session, x_test, y_test
        )

        onnx_int8_diff = float(np.max(np.abs(
            baseline_logits.cpu().numpy() - q_logits
        )))
```

```
        onnx_int8_timing = benchmark_onnx(
            onnx_int8_session, x_test[:1].numpy().astype(np.float32)
        )
        onnx_int8_status = "Success"

    except Exception as error:
        onnx_int8_status = "Failed: " + str(error)[:150]
        print("ONNX INT8 issue:", onnx_int8_status)
else:
    onnx_int8_status = "Quantization API unavailable"

# ---------- 9. PYTORCH MOBILE FP32 ----------

mobile_fp32_path = os.path.join(OUT_DIR, "mobilenet_v2_fp32.ptl")
mobile_fp32_accuracy = None
mobile_fp32_timing = None
mobile_fp32_diff = None
mobile_fp32_status = "Not run"

try:
    print("\nTracing and optimizing PyTorch Mobile FP32...")
    traced = torch.jit.trace(model, dummy_input).eval()

    if MOBILE_OPT_AVAILABLE:
        mobile_model = optimize_for_mobile(traced)
    else:
        mobile_model = traced

    mobile_model._save_for_lite_interpreter(mobile_fp32_path)

    if hasattr(torch.jit, "_load_for_lite_interpreter"):
        lite_model = torch.jit._load_for_lite_interpreter(
```

```
            mobile_fp32_path
        )

        mobile_logits, mobile_fp32_accuracy = evaluate_torch(
            lite_model, x_test, y_test
        )

        mobile_fp32_diff = float(torch.max(torch.abs(
            baseline_logits.cpu() - mobile_logits
        )).item())

        mobile_fp32_timing = benchmark_torch(lite_model, x_test)
        mobile_fp32_status = "Success: Lite inference measured"
    else:
        mobile_fp32_status = (
            "PTL saved; Lite interpreter unavailable in this build"
        )

except Exception as error:
    mobile_fp32_status = "Failed: " + str(error)[:150]

print("PyTorch Mobile FP32:", mobile_fp32_status)

# -------------------- 10. PYTORCH DYNAMIC INT8 --------------------

mobile_int8_path = os.path.join(OUT_DIR, "mobilenet_v2_int8.ptl")
mobile_int8_accuracy = None
mobile_int8_timing = None
mobile_int8_diff = None
mobile_int8_status = "Not run"

try:
```

```
print("\nApplying PyTorch dynamic quantization...")
quantized_model = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
).eval()

quantized_traced = torch.jit.trace(
    quantized_model, dummy_input
).eval()

if MOBILE_OPT_AVAILABLE:
    quantized_traced = optimize_for_mobile(quantized_traced)

quantized_traced._save_for_lite_interpreter(mobile_int8_path)

if hasattr(torch.jit, "_load_for_lite_interpreter"):
    lite_int8 = torch.jit._load_for_lite_interpreter(
        mobile_int8_path
    )

    q_mobile_logits, mobile_int8_accuracy = evaluate_torch(
        lite_int8, x_test, y_test
    )

    mobile_int8_diff = float(torch.max(torch.abs(
        baseline_logits.cpu() - q_mobile_logits
    )).item())

    mobile_int8_timing = benchmark_torch(lite_int8, x_test)
    mobile_int8_status = "Success: Lite inference measured"
else:
    mobile_int8_status = (
        "PTL saved; Lite interpreter unavailable in this build"
```

```
        )

except Exception as error:
    mobile_int8_status = "Failed: " + str(error)[:150]

print("PyTorch Mobile INT8:", mobile_int8_status)

# ---------- 11. COMPARISON TABLE ----------

results = []

def add_row(platform, precision, accuracy, timing, path,
            output_diff=None, status="Success"):

    exists = os.path.exists(path)
    model_size = size_mb(path) if exists else None

    results.append({
        "Platform": platform,
        "Precision": precision,
        "Accuracy_%": (
            round(accuracy, 3) if accuracy is not None else None
        ),
        "Accuracy_Deviation_pp": (
            round(abs(accuracy - baseline_accuracy), 3)
            if accuracy is not None else None
        ),
        "Mean_Latency_ms": (
            round(timing["mean"], 3) if timing is not None else None
        ),
        "Std_Dev_ms": (
            round(timing["std"], 3) if timing is not None else None
```

```
        ),
        "P95_Latency_ms": (
            round(timing["p95"], 3) if timing is not None else None
        ),
        "Throughput_FPS": (
            round(timing["fps"], 2) if timing is not None else None
        ),
        "Model_Size_MB": (
            round(model_size, 3) if model_size is not None else None
        ),
        "Size_Reduction_%": (
            round((1 - model_size / baseline_size) * 100, 2)
            if model_size is not None else None
        ),
        "Max_Output_Difference": (
            output_diff if output_diff is not None else None
        ),
        "Status": status
    })


add_row(
    "PyTorch Baseline", "FP32", baseline_accuracy,
    baseline_timing, baseline_path, 0.0
)

add_row(
    "ONNX Runtime", "FP32", onnx_accuracy,
    onnx_timing, onnx_fp32_path, onnx_diff
)

add_row(
```

```
    "ONNX Runtime", "INT8", onnx_int8_accuracy,
    onnx_int8_timing, onnx_int8_path,
    onnx_int8_diff, onnx_int8_status
)

add_row(
    "PyTorch Mobile", "FP32", mobile_fp32_accuracy,
    mobile_fp32_timing, mobile_fp32_path,
    mobile_fp32_diff, mobile_fp32_status
)

add_row(
    "PyTorch Mobile", "INT8", mobile_int8_accuracy,
    mobile_int8_timing, mobile_int8_path,
    mobile_int8_diff, mobile_int8_status
)

results_df = pd.DataFrame(results)

print("\n" + "=" * 90)
print("CROSS-PLATFORM EDGE AI PERFORMANCE COMPARISON")
print("=" * 90)
display(results_df)

# -------------------- 12. VALIDATION ----------------------------

print("\nVALIDATION AGAINST EXPERIMENT THRESHOLDS")
print("-" * 60)
print("ONNX graph check: PASS")
print(
    "ONNX FP32 output difference <= 1e-4:",
    "PASS" if onnx_diff <= 1e-4 else "FAIL"
```

```
)
print(
    "ONNX FP32 accuracy deviation <= 0.5 percentage points:",
    "PASS" if abs(onnx_accuracy - baseline_accuracy) <= 0.5 else "FAIL"
)

if mobile_fp32_accuracy is not None:
    print(
        "Mobile FP32 accuracy deviation <= 0.5 pp:",
        "PASS" if abs(mobile_fp32_accuracy - baseline_accuracy) <= 0.5
        else "FAIL"
    )
else:
    print("Mobile FP32 accuracy: NOT MEASURED")

if onnx_int8_accuracy is not None:
    print(
        "ONNX INT8 accuracy deviation <= 2 pp:",
        "PASS" if abs(onnx_int8_accuracy - baseline_accuracy) <= 2
        else "FAIL"
    )
else:
    print("ONNX INT8 accuracy: NOT MEASURED")

if mobile_int8_accuracy is not None:
    print(
        "Mobile INT8 accuracy deviation <= 2 pp:",
        "PASS" if abs(mobile_int8_accuracy - baseline_accuracy) <= 2
        else "FAIL"
    )
else:
    print("Mobile INT8 accuracy: NOT MEASURED")
```

```
# -------------------- 13. GRAPHS --------------------------------

plot_df = results_df.dropna(subset=["Mean_Latency_ms"])

if not plot_df.empty:
    plot_labels = plot_df["Platform"] + " " + plot_df["Precision"]

    plt.figure(figsize=(10, 5))
    plt.bar(plot_labels, plot_df["Mean_Latency_ms"])
    plt.title("Mean Inference Latency Comparison")
    plt.ylabel("Latency (ms)")
    plt.xticks(rotation=25, ha="right")
    plt.grid(axis="y")
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(10, 5))
    plt.bar(plot_labels, plot_df["Model_Size_MB"])
    plt.title("Model File Size Comparison")
    plt.ylabel("Size (MB)")
    plt.xticks(rotation=25, ha="right")
    plt.grid(axis="y")
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(10, 5))
    plt.bar(plot_labels, plot_df["Accuracy_%"])
    plt.title("Classification Accuracy Comparison")
    plt.ylabel("Accuracy (%)")
    plt.xticks(rotation=25, ha="right")
    plt.grid(axis="y")
```

```
    plt.tight_layout()
    plt.show()

# -------------------- 14. EXPORT RESULTS -------------------------

csv_path = os.path.join(OUT_DIR, "edge_ai_comparison.csv")
txt_path = os.path.join(OUT_DIR, "experiment_summary.txt")

results_df.to_csv(csv_path, index=False)

with open(txt_path, "w") as f:
    f.write("Cross-Platform Edge AI Deployment Experiment\n")
    f.write(f"PyTorch: {torch.__version__}\n")
    f.write(f"ONNX: {onnx.__version__}\n")
    f.write(f"ONNX Runtime: {ort.__version__}\n")
    f.write(f"Test samples: {len(x_test)}\n")
    f.write(f"Warm-up runs: {WARMUP_RUNS}\n")
    f.write(f"Timed runs: {TIMED_RUNS}\n")
    f.write(f"CPU intra-op threads: {torch.get_num_threads()}\n")
    f.write(f"Baseline accuracy: {baseline_accuracy:.3f}%\n")
    f.write(f"ONNX output difference: {onnx_diff:.8g}\n\n")
    f.write(results_df.to_string(index=False))
    f.write("\n\nPyTorch Mobile status: " + mobile_fp32_status)
    f.write("\nPyTorch Mobile INT8 status: " + mobile_int8_status)
    f.write(
        "\n\nResults are specific to the Colab CPU and selected dataset. "
        "A .ptl file is not considered successfully tested unless Lite "
        "interpreter inference actually ran."
    )

print("\nExperiment finished!")
print("CSV results:", csv_path)
```

```
print("Text summary:", txt_path)
print("All model files:", OUT_DIR)

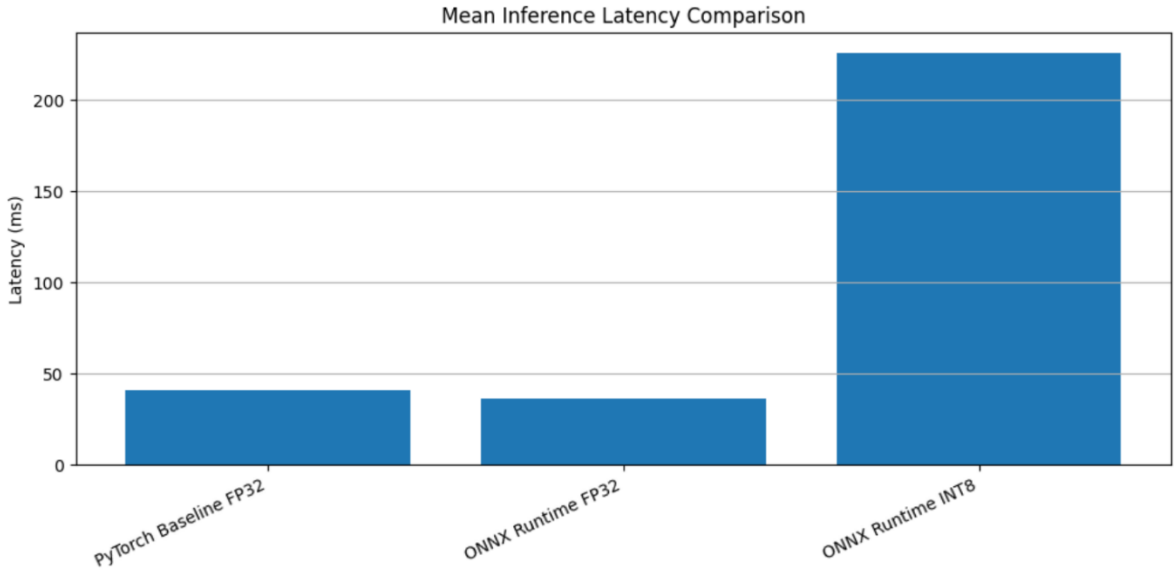
try:
    from google.colab import files
    files.download(csv_path)
    files.download(txt_path)
except Exception:
    print("Download manually from the Colab Files panel.")

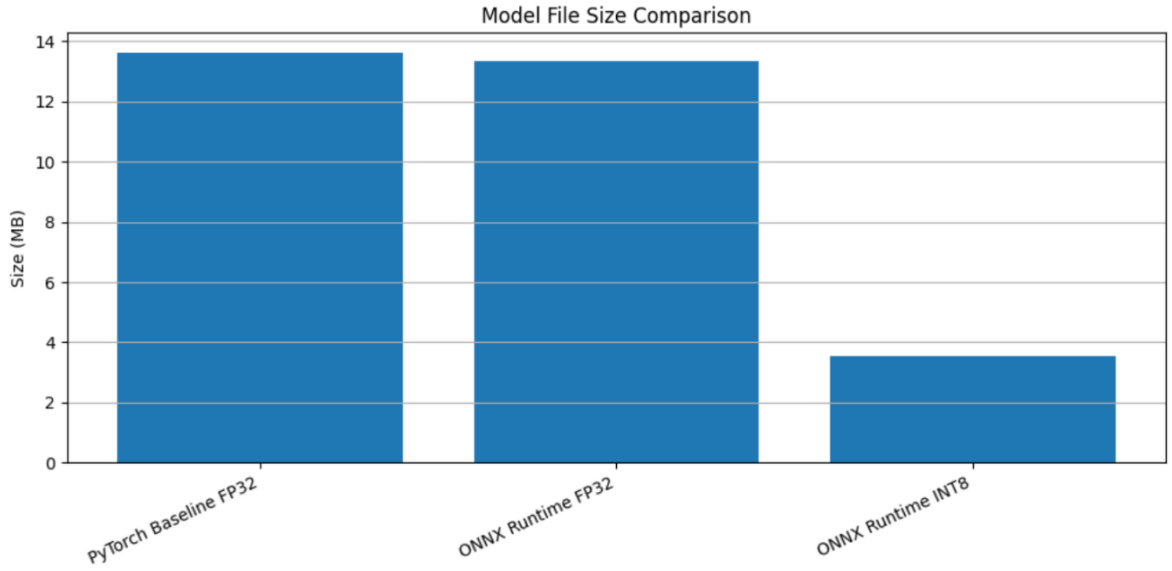
print(
    "\nInterpretation: ONNX Runtime is useful for cross-framework "
    "deployment and gateways. PyTorch Mobile is intended for mobile "
    "deployment, but this runtime must support Lite Interpreter inference "
    "before its performance can be compared fairly."
)
```

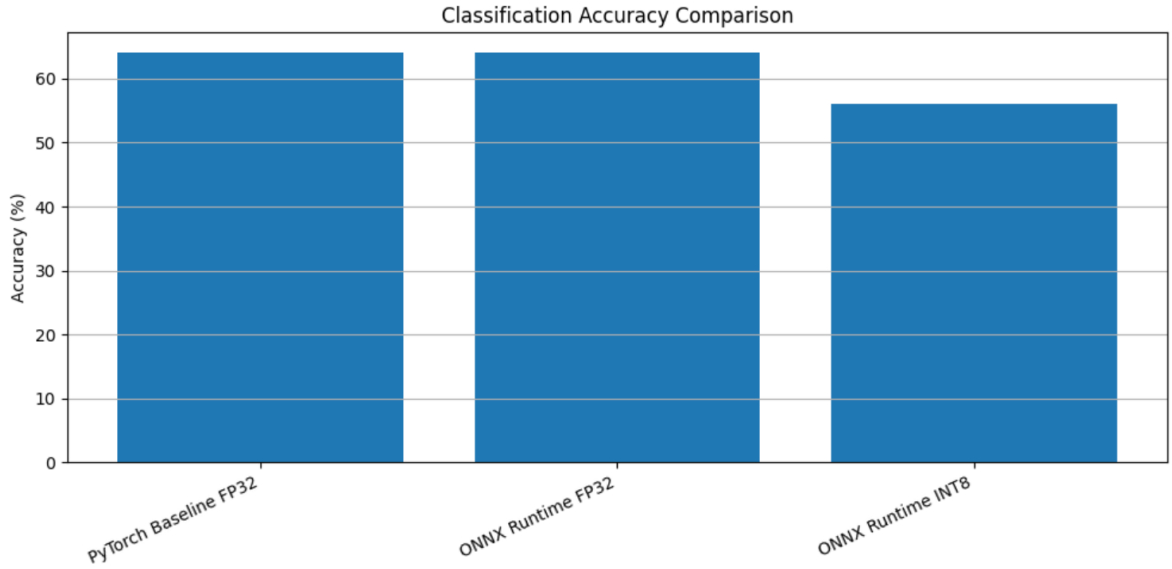
```
================================================================================
CROSS-PLATFORM EDGE AI PERFORMANCE COMPARISON
================================================================================
```

	Platform	Precision	Accuracy_%	Accuracy_Deviation_pp	Mean_Latency_ms	Std_Dev_ms	P95_Latency_ms	Throughput_FPS	Model_Size_MB	Size_Reduction_%	Max_Output_
0	PyTorch Baseline	FP32	64.0	0.0	40.842	6.551	51.156	24.48	13.602	0.00	
1	ONNX Runtime	FP32	64.0	0.0	36.453	9.276	47.490	27.43	13.344	1.90	
2	ONNX Runtime	INT8	56.0	8.0	225.558	87.632	443.345	4.43	3.522	74.11	
3	PyTorch Mobile	FP32	NaN	NaN	NaN	NaN	NaN	NaN	13.272	2.43	
4	PyTorch Mobile	INT8	NaN	NaN	NaN	NaN	NaN	NaN	10.831	20.37	

```
VALIDATION AGAINST EXPERIMENT THRESHOLDS
----------------------------------------------------------------
ONNX graph check: PASS
ONNX FP32 output difference <= 1e-4: PASS
ONNX FP32 accuracy deviation <= 0.5 percentage points: PASS
Mobile FP32 accuracy: NOT MEASURED
ONNX INT8 accuracy deviation <= 2 pp: FAIL
Mobile INT8 accuracy: NOT MEASURED
```


Mean Inference Latency Comparison
Latency (ms)
0
50
100
150
200
PyTorch Baseline FP32
ONNX Runtime FP32
ONNX Runtime INT8


Model File Size Comparison
Size (MB)
0
2
4
6
8
10
12
14
PyTorch Baseline FP32
ONNX Runtime FP32
ONNX Runtime INT8


Classification Accuracy Comparison
Accuracy (%)
0
10
20
30
40
50
60
PyTorch Baseline FP32
ONNX Runtime FP32
ONNX Runtime INT8